

birdclef-2026

# 5th Place Solution: Diversity and Bug - Both Are All You Need

|          |             |
|----------|-------------|
| Rank     | 5           |
| Team     | Jiacheng Ma |
| Author   | 马家☒         |
| Topic ID | 704602      |
| Version  | Original    |

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704602>

Retrieved with Kaggle CLI on 2026-07-03.

First, thank you to the sponsor and Kaggle for this wonderful event. The high-quality data and fair environment made this competition challenging and rewarding.

I'm deeply grateful to all participants who shared insights, notebooks, and discussions. The collaborative spirit of this community is truly inspiring — I learned so much from you, and this solution wouldn't exist without those exchanges.

This is my first gold medal, and I'm still overwhelmed. Reaching the top took real luck — fortunate validation splits, lucky ensemble choices, and timing that worked in my favour. I also recognize that many excellent competitors achieved higher scores on private submissions but didn't select them as final. Had they done so, the leaderboard would look very different.

Finally, I simply got lucky along the way. Sometimes things just click, and this was one of those rare moments. Thank you all for an unforgettable competition.

## Data

Competition **focal + soundscapes**. In my architecture, just simply concatenating competition focal and soundscapes data yielded the best results. I once tried to find the correlation between CV and LB, but ultimately failed. I only selected models based on the validation set AUC and loss split from a portion of the focal data, with all labeled soundscapes used for training.

## Augmentation

- **Mixup**: Audio-level mix with max-lambda strategy ( $\lambda = \max(\beta, 1-\beta)$  where  $\beta \sim \text{Beta}(\alpha, \alpha)$ ), ensuring the original sample dominates the mixture ( $\lambda \geq 0.5$ ).
- **Filter\_aug**: applies random piecewise-linear gain across frequency bands, simulating spectral coloration.
- **Wave-noisy**: gain jitter, additive noise, and random shift.
- **SpecAugment** (time masking only): randomly masks contiguous time steps in the mel-spectrogram.

## Model Families

All cnn models based on a public baseline: [BirdCLEF+2026: HGNetV2-B0 Baseline \[Training\]](#) (Thanks to [@ttahara](#)) with the original SED head replaced by another design. And finally all of them based on perch-distilled.

| backbone  | pretrain | lms_shape | duration | loss | upsampling     | perch-distilled | self-distilled | pseudo   | fold | seed A/B |
|-----------|----------|-----------|----------|------|----------------|-----------------|----------------|----------|------|----------|
| hgnetv2b0 | -        | 128, 313  | 5s       | bce  | $\sqrt{\quad}$ | $\sqrt{\quad}$  | $\times$       | $\times$ | 0    | 520/3407 |
|           |          |           | 5s       |      | $\times$       | $\sqrt{\quad}$  | 2-iter         | 1-iter   | 0    |          |

| backbone        | pretrain                                             | lms_shape | duration | loss | upsampling | perch-distilled | self-distilled | pseudo | fold | seed A/B |
|-----------------|------------------------------------------------------|-----------|----------|------|------------|-----------------|----------------|--------|------|----------|
| efficientnetv2s | <a href="#">bird-clef-2025-all-pretrained-models</a> | 128, 313  |          | bce  |            |                 |                |        |      | 1086/42  |
| efficientnetv2s | <a href="#">bird-clef-2025-all-pretrained-models</a> | 128, 626  | 10s      | bce  | ×          | √               | ×              | ×      | 0    | 1086/1   |
| efficientnetb3  | -                                                    | 384, 384  | 5s       | bce  | ×          | √               | ×              | 1-iter | 0    | 1086/888 |

For each of my single model, aside from hyperparameter tuning, the greatest gains came from: **Distillation**(50%), **various data augmentations**(30%), **model architecture**(20%). For my best single model, some of its details are as follows:

| Improve           | Public Score |
|-------------------|--------------|
| Baseline          | 0.88-0.895   |
| + Data Aug        | 0.895-0.905  |
| + Perch-Distilled | 0.905-0.915  |
| + Pretrained      | 0.915-0.925  |
| + Self-Distilled  | 0.925-0.928  |
| + Pseudo iter 1   | 0.928-0.935  |
| + TTA             | +0.005-0.008 |
| Post-Processing   | +0.005-0.01  |

Special thanks to [2nd Place. Journey Down the Rabbit Hole of Pseudo Labels](#) and [5th place solution: Self-Distillation is All You Need](#) for their outstanding solutions from last year's competition, which heavily influenced my ensemble and pseudo-labeling strategies.

## Post-Processing

- **TTA:** Dual-window inference (normal + 2.5s shift).
- **Smoothing:** Applied smoothing using neighboring frames with a window of [0.1, 0.8, 0.1]([last year 5th place solution](#)).
- **File Peak Scale:** Scales predictions within each audio file by the mean of top-k window probabilities([last year 2th place solution](#)).

# Final Ensemble

- **CNN ensemble:** hgnetv2-b0 + efficientnetv2-s + efficientnet-b3 with SED Head. Inference only use clipwise.
- **Sequential model:** ProtoSSM based on public model(Thanks to the people for sharing them)
- **Distilled SED Baseline**(Thanks to [@tuckerarrants](#))

```
Final = 0.6 · (0.25 · hgnet + 0.40 · effv2_5s + 0.25 · effv2_10s + 0.10 · effb3) + 0.4 · (0.6 · proto + 0.4 · sed)
```

To maximize diversity, I minimized the Mel parameters as much as possible and optimized single model inference to make it faster, allowing me to ensemble more models and reduce variance. So I mean **Diversity Is All You Need!** One lesson learned: applying the same post-processing to mutil models actually hurts LB. So in my final submission, I applied **different post-processing at different positions** instead.

## Bug

So far, as described above, my work has been fairly conventional, and even the pseudo-labeling part was not particularly well executed. The night before the competition ended, I was still worried that my model might suffer from significant score fluctuations or drops, because my data processing was not very robust and the data diversity itself was not strong. Therefore, when I saw how well my model performed on the private leaderboard the next day, I was somewhat surprised myself — it fit the private leaderboard surprisingly well.

| Submission and Description                                                                                                                     | Private Score ⓘ | Public Score ⓘ | Selected                            |
|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|----------------|-------------------------------------|
|  <b>PayOff - Version 1</b><br>Succeeded · 2d ago · Paid Off | 0.95824         | 0.95715        | <input checked="" type="checkbox"/> |
|  <b>ensemble2post - Version 6</b><br>Succeeded · 2d ago     | 0.95713         | 0.95712        | <input type="checkbox"/>            |
|  <b>ensemble2post - Version 5</b><br>Succeeded · 2d ago     | 0.95765         | 0.95710        | <input type="checkbox"/>            |
|  <b>ensemble2post - Version 4</b><br>Succeeded · 2d ago     | 0.95715         | 0.95714        | <input type="checkbox"/>            |
|  <b>ensemble2post - Version 3</b><br>Succeeded · 2d ago     | 0.95715         | 0.95714        | <input checked="" type="checkbox"/> |
|  <b>ensemble2post - Version 2</b><br>Succeeded · 3d ago     | 0.95705         | 0.95701        | <input type="checkbox"/>            |

So I started exploring the specific reasons and eventually discovered a processing detail that I call a **"bug"**. In data augmentation, the conventional FrequencyFilterAug is essentially `dB + noise` (addition/subtraction in the dB domain), whereas my approach was `dB × linear_gain` (multiplication in the dB domain). From a physical audio processing perspective, multiplying Log-Mel completely destroys the physical meaning of the mel spectrogram — a quiet frequency band originally at -80dB, when multiplied by 2 (equivalent to a 6dB gain), becomes -160dB, which is physically absurd. Yet, this "crazy scaling in the dB domain" multiplication

operation unexpectedly brought excellent generalization. Through my partial testing, this method delivered a significant improvement of 0.01+ on my model. Similar to the "God Trick" below:

```
def freq_filt_aug(features, db_range=(-6, 6), n_band=(3, 6), min_bw=6):
    B, n_freq, _ = features.shape
    n_bands = torch.randint(n_band[0], n_band[1], (1,)).item()
    if n_bands <= 1:
        return features
    while n_freq - n_bands * min_bw + 1 < 0:
        min_bw -= 1
    bndry = torch.sort(
        torch.randint(0, n_freq - n_bands * min_bw + 1, (n_bands - 1,))
    )[0] + torch.arange(1, n_bands) * min_bw
    bndry = torch.cat([torch.tensor([0]), bndry, torch.tensor([n_freq])])
    factors = torch.rand((B, n_bands + 1), device=features.device) * (db_range[1] - db_range[0]) +
    db_range[0]
    freq_filt = torch.ones((B, n_freq, 1), device=features.device)
    for i in range(n_bands):
        l, r = int(bndry[i].item()), int(bndry[i + 1].item())
        for j in range(B):
            freq_filt[j, l:r, :] = torch.linspace(
                factors[j, i].item(), factors[j, i + 1].item(), r - l,
                device=features.device,
            ).unsqueeze(-1)
    return features * (10 ** (freq_filt / 10))
```

## Closing

I'm still new to writing solution posts, so please excuse any imperfections. Any suggestions or advice are greatly appreciated — I'm always eager to learn from this wonderful community. Once again, thank you all — this community is truly special. 😊

## Resources

- Inference Kernel: [5th-solution notebook](#)
- Final Inference Weights: [birdclef2026-5th-final-cnn-models](#)
- Github Repo: [BirdCLEF2026-5th-solution](#)